# COMPUTER VISION

# Assignment 2 — Complete Solutions

B.Tech Level • Questions + Answers • Equations • Numerical Example • Diagrams

**Contents**

Q1. Digital Image Formation and Digital Image Processing
Q2. Kernel, Padding, Stride and Convolution
Q3. Intensity Transformations and Histogram Stretching
Q4. Affine, Geometric, Orthogonal and Euclidean Transformations
Q5. Image Pre-processing Pipeline
Q6. Computer Vision Architecture
Q7. Two-Dimensional Fourier Transform and Frequency Filtering
Q8. Image Enhancement vs Restoration and Noise/Blur Removal

Prepared as an assignment-ready reference. The question wording and solution content are based on the supplied Assignment 2 material.

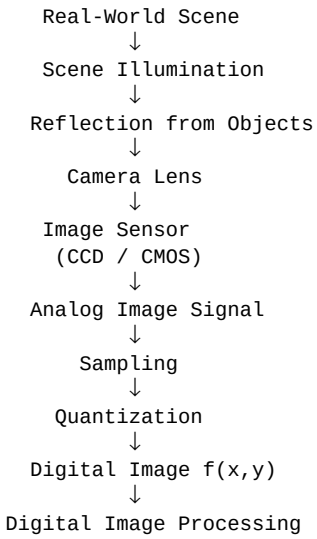
# Computer Vision — Assignment 2

## Q1. Explain how a digital image is formed, starting from scene illumination and image sensing to sampling and quantization. Also describe the major steps involved in digital image processing.

### Answer

A **digital image** is formed when light from a real-world scene is captured by an imaging sensor and converted into a numerical representation.

The complete process is:




### 1. Scene Illumination

A light source illuminates the objects in the scene.

Examples:

- Sunlight
- LED
- Bulb
- Flash

The objects reflect part of this incident light toward the camera.

The observed brightness depends on both **illumination** and **surface reflectance**.

A simple model is:

$$f(x,y)=i(x,y)r(x,y)$$

where:

- (i(x,y)) = illumination component
- (r(x,y)) = reflectance component
- (f(x,y)) = observed image intensity

### 2. Image Sensing

The reflected light enters the camera through the lens.

The lens focuses the light onto an image sensor such as:

- **CCD (Charge-Coupled Device)**
- **CMOS (Complementary Metal-Oxide Semiconductor)**

The sensor converts light energy into an electrical signal.

```
Object
  ↓ reflected light
■■■■■■■■■■■
■  Lens  ■
■■■■■■■■■■■
    ↓
■■■■■■■■■■■■■■■■
■ CMOS Sensor ■
■■■■■■■■■■■■■■■■
    ↓
Analog Signal
```

## 3. Sampling

The sensor initially represents the image continuously.

**Sampling** converts the continuous spatial coordinates into discrete pixels.

If an image has (M) rows and (N) columns:

```
f(x,y) arrow f[m,n]
```

The image becomes an (M\times N) pixel matrix.

For example:

```
[ 10&20&30 40&50&60 70&80&90 ]
```

Each element represents one pixel.

## 4. Quantization

Sampling makes the spatial coordinates discrete, while **quantization makes intensity values discrete**.

For an 8-bit grayscale image:

```
L=2^8=256
```

Therefore:

```
0 ≤ f(x,y) ≤ 255
```

where:

- 0 → black
- 255 → white
- intermediate values → gray levels

## Major Steps in Digital Image Processing

```
Image Acquisition
       ↓
Pre-processing
       ↓
Segmentation
       ↓
Representation
       ↓
Feature Extraction
       ↓
Classification / Recognition
       ↓
Output
```

## 1. Image Acquisition

Capturing the image using a camera or scanner.

## 2. Image Enhancement

Improving the visual appearance of an image.

Examples:

- Contrast enhancement
- Sharpening
- Histogram equalization

## 3. Image Restoration

Removing known or modeled degradations such as:

- Blur
- Noise
- Motion distortion

## 4. Segmentation

Dividing an image into meaningful regions.

Example:

```
Image → Background + Object
```

## 5. Feature Extraction

Extracting useful information such as:

- Edges
- Corners
- Texture
- Shape
- Color

## 6. Classification/Recognition

Identifying the object or class.

Example:

```
Image arrow Cat
```

## Conclusion

Digital image formation involves **illumination, sensing, sampling, and quantization**. The resulting digital image can then undergo various processing operations such as enhancement, restoration, segmentation, feature extraction, and recognition.

## Q2. Explain the concepts of kernel, padding, and stride in image convolution. Demonstrate their effect using a suitable numerical example and determine the output dimensions.

### Answer

**Convolution** is an important operation in computer vision and is widely used in image filtering and CNNs.

A small matrix called a **kernel/filter** is moved over the image to calculate a new output image.

## 1. Kernel

A **kernel** is a small matrix containing weights.

For example, a (3\times3) kernel:

```
K= [ 1&0&1 0&1&0 1&0&1 ]
```

The kernel slides over the image and performs multiplication and addition.

## 2. Padding

**Padding** means adding extra pixels around the boundary of an image before convolution.

Common padding:

### Valid Padding

No extra pixels are added.

### Same Padding

Padding is added so that the output can have approximately the same spatial size as the input.

For a (3\times3) kernel, commonly:

```
P=1
```

is used for stride 1.

## 3. Stride

**Stride** specifies how many pixels the kernel moves after each operation.

- Stride (=1): moves one pixel
- Stride (=2): moves two pixels

Larger stride produces a smaller output.

## Numerical Example

Consider the input image:

```
I= [ 1&2&3&4 5&6&7&8 9&10&11&12 13&14&15&16 ]
```

and kernel:

```
K= [ 1&0 0&1 ]
```

### Case 1: No Padding, Stride = 1

Input:

$$4 \times 4$$

Kernel:

$$2 \times 2$$

Output dimensions:

$$O=\lfloor (N-F+2P)/(S) \rfloor+1$$

where:

- $(N=4)$
- $(F=2)$
- $(P=0)$
- $(S=1)$

Therefore:

$$O=(4-2+0)/(1)+1=3$$

So output is:

$$3 \times 3$$

For the first position:

$$(1)(1)+(2)(0)+(5)(0)+(6)(1)$$

$$=1+6=7$$

The complete output is:

$$[\ 7\&9\&11\ 15\&17\&19\ 23\&25\&27\ ]$$

## Case 2: Stride = 2

$$O=\lfloor (4-2)/(2) \rfloor+1$$

$$=2$$

Therefore output size:

$$2 \times 2$$

Output:

$$[\ 7\&11\ 23\&27\ ]$$

So increasing the stride reduces the spatial dimensions.

## Case 3: Padding = 1, Stride = 1

Now:

$$P=1$$

Therefore:

$$O= \lfloor (4-2+2(1))/(1) \rfloor+1$$

$$=5$$

Thus the output becomes:

$$5 \times 5$$

## General Output Formula

For an input of $(N\times N)$:

```
O= lfloor (N-F+2P)/(S) rfloor+1
```

For different height and width:

```
H_out= lfloor (H-F_h+2P)/(S) rfloor+1
```

```
W_out= lfloor (W-F_w+2P)/(S) rfloor+1
```

### Summary

| ParameterMeaningEffect | | |
|---|---|---|
| Kernel | Matrix used for filtering | Extracts local information |
| Padding | Extra border pixels | Controls boundary/output size |
| Stride | Kernel movement | Larger stride → smaller output |

## Q3. Explain any two linear and any two non-linear intensity transformation techniques. How does histogram-based processing improve image contrast? Illustrate histogram stretching with a suitable example.

### Answer

Intensity transformation changes the intensity value of each pixel:

```
s=T(r)
```

where:

- (r) = input intensity
- (s) = output intensity
- (T) = transformation function

## A. Linear Intensity Transformations

### 1. Image Negative

The negative transformation reverses the intensity values.

For an image with maximum intensity (L-1):

```
s=(L-1)-r
```

For an 8-bit image:

```
s=255-r
```

Example:

```
r=50
```

```
s=255-50=205
```

This transformation is useful when bright regions need to be converted to dark regions and vice versa.

### 2. Contrast Stretching

Contrast stretching expands a narrow range of intensity values into a wider range.

General formula:

```
s= fracr-r_minr_max-r_min (L-1)
```

For an 8-bit image:

$$L-1=255$$

So:

$$s= fracr-r_minr_max-r_min \times 255$$

It makes dark pixels darker and bright pixels brighter, improving contrast.

# B. Non-Linear Intensity Transformations

## 1. Logarithmic Transformation

The log transformation is:

$$s=clog(1+r)$$

where (c) is a scaling constant.

It expands dark intensity values and compresses bright values.

It is useful for images where important details are present in dark regions.

## 2. Gamma Transformation

Also called **power-law transformation**:

$$s=cr^\gamma$$

where:

- (c) = constant
- (\gamma) = gamma value

If:

$$\gamma<1$$

the image becomes brighter.

If:

$$\gamma>1$$

the image becomes darker.

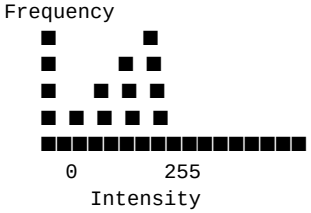
# Histogram-Based Processing

An image histogram represents the frequency of each intensity level.

For an 8-bit grayscale image:

$$0 \leq r \leq 255$$

The histogram can be represented as:




If most pixels occupy a small intensity range, the image has **low contrast**.

Histogram-based processing redistributes pixel intensities to improve contrast.

## Histogram Stretching Example

The matrix given in the assignment is:

```
[ 4&1&3&2 3&1&1&1 0&1&5&2 1&1&2&2 ]
```

The minimum intensity is:

```
r_min=0
```

The maximum intensity is:

```
r_max=5
```

Assume we want to stretch the values to the range:

```
0 to 255
```

Using:

```
s= fracr-r_minr_max-r_min × 255
```

```
Since (r_min=0) and (r_max=5):
```

```
s=(r)/(5) × 255
```

Thus:

| Input (r)Output (s) | |
|---|---|
| 0 | 0 |
| 1 | 51 |
| 2 | 102 |
| 3 | 153 |
| 4 | 204 |
| 5 | 255 |

Therefore, the stretched image is:

```
[ 204&51&153&102 153&51&51&51 0&51&255&102 51&51&102&102 ]
```

### Result

Before stretching, the image uses only intensity values:

```
0,1,2,3,4,5
```

After stretching, the values are spread across:

```
0,51,102,153,204,255
```

Therefore, the **dynamic range increases**, resulting in better image contrast.

## Q4. Compare affine transform with geometric, orthogonal and Euclidean transform. Also write their mathematical representation.

### Answer

Geometric transformations modify the position, orientation, scale, or shape of objects in an image.

A general transformation can be represented as:

```
mathbfx'=T(mathbfx)
```

## 1. Affine Transformation

An affine transformation preserves:

- straight lines
- parallel lines

But it may change:

- angles
- lengths
- scale

Mathematical representation:

## [ \boxed{ \begin{bmatrix} x'\ y' \end{bmatrix}

[

a&b\\

c&d

]

[

x\\

y

]

+

[

t_x\\

t_y

]

]

In homogeneous coordinates:

## [ \boxed{ \begin{bmatrix} x'\y'\1 \end{bmatrix}

[

a&b&t_x\\

c&d&t_y\\

0&0&1

]

[

x\y\1

]

]

Examples:

- Rotation
- Scaling
- Translation
- Shearing

## 2. Euclidean Transformation

Euclidean transformation preserves:

- distances
- angles
- shape

It consists mainly of:

- rotation
- translation

Mathematical representation:

```
mathbfx'=Rmathbfx+mathbft
```

where:

```
R= [ cosθ&-sinθ sinθ&cosθ ]
```

## 3. Orthogonal Transformation

An orthogonal transformation uses an orthogonal matrix:

```
Q^TQ=I
```

Therefore:

```
mathbfx'=Qmathbfx
```

Orthogonal transformations preserve:

- lengths
- angles
- dot products

Examples include:

- rotation
- reflection

## 4. Geometric Transformation

Geometric transformation is the general process of changing the spatial coordinates of an image.

Examples:

- Translation
- Rotation
- Scaling
- Reflection

- Shearing
- Affine transformation

### Comparison

| TransformationPreserves LengthPreserves AnglePreserves Parallel Lines | | | |
|---|---|---|---|
| Affine | No | No | Yes |
| Euclidean | Yes | Yes | Yes |
| Orthogonal | Yes | Yes | Yes |
| General geometric | Depends on transformation | Depends | Depends |

## Q5. Design and explain an image pre-processing pipeline for reducing noise and improving image visibility. Justify the order of operations and the selection of suitable filters and enhancement techniques.

### Answer

Image preprocessing prepares a raw image for further computer vision tasks.

A suitable pipeline is:

```
Input Image
     ↓
Image Acquisition
     ↓
Noise Removal
     ↓
Contrast Enhancement
     ↓
Sharpening
     ↓
Normalization
     ↓
Segmentation / Features
     ↓
Final Analysis
```

## Step 1: Input Image

The image is acquired from a camera or other imaging device.

The image may contain:

- Gaussian noise
- salt-and-pepper noise
- low contrast
- blur

## Step 2: Noise Reduction

The filter should be selected according to the type of noise.

### Gaussian Noise

Use a **Gaussian filter**:

```
G(x,y)= (1)/(2πσ^2) e^-(x^2+y^2)/(2σ^2)
```

It performs smoothing and reduces random Gaussian noise.

### Salt-and-Pepper Noise

Use a **median filter**.

Example:

```
Before:
10  11  12
13 255  14
15  16  17

After median filtering:
10  11  12
13  14  14
15  16  17
```

The median filter removes isolated extreme values while preserving edges better than simple averaging.

## Step 3: Contrast Enhancement

After noise reduction, improve visibility using:

- Histogram stretching
- Histogram equalization
- CLAHE

Histogram equalization redistributes intensity values to improve contrast.

## Step 4: Sharpening

After smoothing and contrast enhancement, sharpening can emphasize edges.

A common Laplacian operator is:

```
∇^2f= (partial^2f)/(partial x^2) + (partial^2f)/(partial y^2)
```

A sharpened image can be obtained as:

```
g(x,y)=f(x,y)-∇^2f(x,y)
```

## Step 5: Normalization

Pixel values can be normalized to a standard range such as:

```
[0,1]
```

using:

```
I_norm= fracI-I_min I_max-I_min
```

### Why This Order?

The order is important.

**Noise removal comes first** because enhancement and sharpening can also amplify noise.

Then:

```
Denoising arrow Enhancement arrow Sharpening
```

This produces a cleaner and more visible image.

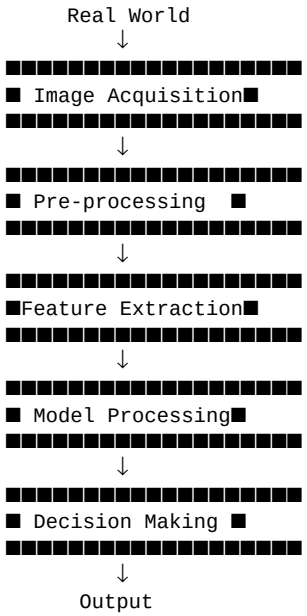
## Q6. Define computer vision and explain its basic architecture. Describe the role of image acquisition, pre-processing, feature extraction, model processing, and decision-making in a computer vision system.

### Answer

### Definition

**Computer Vision** is a field of artificial intelligence and computer science that enables computers to acquire, process, analyze, and understand information from images and videos.

A basic computer vision architecture is:




### 1. Image Acquisition

Captures images using:

- Cameras
- Scanners
- Sensors
- Satellites

Example:

A CCTV camera captures a person's image.

### 2. Pre-processing

Improves the quality of the input image.

Operations include:

- Noise removal
- Resizing
- Contrast enhancement
- Normalization

- Filtering

### 3. Feature Extraction

Extracts meaningful information from the image.

Traditional features include:

- Edges
- Corners
- Texture
- Shape
- Color

Modern deep learning systems can automatically learn features using CNNs.

### 4. Model Processing

A trained model analyzes the extracted information.

Examples:

- CNN
- SVM
- Decision Tree
- YOLO
- Neural Networks

### 5. Decision Making

The system uses the model output to make a decision.

Example:

```
ImagearrowFeaturearrowModelarrowDog
```

For object detection:

```
Imagearrow Person detected
```

## Q7. Explain the role of the two-dimensional Fourier Transform in image processing. Differentiate between low-frequency and high-frequency components, and discuss how frequency-domain filtering is used for image smoothing and sharpening.

### Answer

The **2D Fourier Transform** converts an image from the **spatial domain** to the **frequency domain**.

In the spatial domain, we work directly with pixel intensities.

In the frequency domain, we analyze how rapidly intensity changes across the image.

## 2D Fourier Transform

For an (M\times N) image:

```
F(u,v)= Σ_x=0^M-1 Σ_y=0^N-1 f(x,y) e^-j2π ( (ux)/(M)+(vy)/(N) )
```

where:

- (f(x,y)) = input image
- (F(u,v)) = frequency-domain representation
- (u,v) = frequency coordinates

## Low-Frequency Components

Low frequencies represent slowly changing intensity information.

They generally correspond to:

- Smooth regions
- Overall brightness
- Large-scale structures
- Gradual intensity changes

Example:

```
Smooth region → Low frequency
```

## High-Frequency Components

High frequencies correspond to rapid changes in intensity.

They generally represent:

- Edges
- Fine details
- Texture
- Noise

Example:

```
Sharp edge → High frequency
```

## Frequency-Domain Filtering

The filtering process is:

```
Image f(x,y)
     ↓
 Fourier Transform
     ↓
   F(u,v)
     ↓
Apply Filter H(u,v)
     ↓
G(u,v)=H(u,v)F(u,v)
     ↓
Inverse Fourier Transform
     ↓
Filtered Image
```

Mathematically:

$$G(u,v)=H(u,v)F(u,v)$$

Then:

$$g(x,y)=F^{-1}G(u,v)$$

## Low-Pass Filtering

A **low-pass filter (LPF)** allows low frequencies and suppresses high frequencies.

Therefore:

```
Low-pass filtering arrow Smoothing
```

It can reduce:

- Noise
- Fine details
- Rapid intensity changes

Examples:

- Gaussian low-pass filter
- Ideal low-pass filter

## High-Pass Filtering

A **high-pass filter (HPF)** allows high frequencies and suppresses low frequencies.

Therefore:

```
High-pass filtering arrow Sharpening
```

It enhances:

- Edges
- Fine details
- Texture

### Comparison

| FeatureLow FrequencyHigh Frequency | | |
|---|---|---|
| Represents | Smooth regions | Edges/details |
| Intensity change | Slow | Rapid |
| Filtering | Low-pass | High-pass |
| Main application | Smoothing | Sharpening |
| Noise | Usually suppressed | May be enhanced |

# Q8. Differentiate between image enhancement and image restoration. Explain how blur, Gaussian noise, and salt-and-pepper noise affect an image, and suggest suitable restoration or filtering methods for each case.

### Answer

Image enhancement and image restoration are related but different techniques.

## Image Enhancement

**Image enhancement** improves the visual appearance or usefulness of an image.

It does not necessarily require a mathematical model of the degradation.

Examples:

- Histogram equalization
- Contrast stretching
- Sharpening
- Smoothing

## Image Restoration

**Image restoration** attempts to recover the original image from a degraded image using a mathematical model of the degradation.

General degradation model:

```
g(x,y)=h(x,y)*f(x,y)+n(x,y)
```

where:

- (f(x,y)) = original image
- (h(x,y)) = degradation/blur function
- (n(x,y)) = noise
- (g(x,y)) = degraded image
- (*) = convolution

## Difference

| Image EnhancementImage Restoration | |
|---|---|
| Improves visual quality | Attempts to recover original image |
| Subjective | More objective/model-based |
| Does not necessarily know degradation | Uses degradation model |
| Histogram equalization | Wiener filtering |
| Contrast stretching | Inverse filtering |
| Sharpening | Deblurring |

## Types of Image Degradation

## 1. Blur

Blur occurs when image details become spread out.

Causes include:

- Camera motion
- Defocusing

- Atmospheric effects

Mathematically:

```
g(x,y)=h(x,y)*f(x,y)
```

### Suitable Methods

For blur:

- Inverse filtering
- Wiener filtering
- Deblurring
- Regularized filtering

### Wiener Filter

The Wiener filter tries to minimize the mean-square error between the estimated and original image.

It is especially useful when both blur and noise are present.

## 2. Gaussian Noise

Gaussian noise is random noise with a Gaussian probability distribution.

Its probability density function is:

```
p(z)= frac1√2πσ^2 e^-((z-μ)^2)/(2σ^2)
```

where:

- (\mu) = mean
- (\sigma^2) = variance

It appears as small random intensity variations throughout the image.

### Suitable Filters

For Gaussian noise:

- Gaussian filter
- Mean filter
- Wiener filter

The **Wiener filter** is particularly useful when the noise characteristics are known or can be estimated.

## 3. Salt-and-Pepper Noise

Salt-and-pepper noise appears as random:

- white pixels → salt
- black pixels → pepper

Example:

```
Normal image:

100 100 100
100 100 100
100 100 100

With salt-and-pepper noise:
```

```
100   0   100
255 100   100
100 100    0
```

### Suitable Filter

The best common choice is the:

```
Median Filter
```

It replaces each pixel with the median value of its neighborhood.

It is effective because it removes isolated extreme values while preserving edges better than averaging.

## Final Comparison

| DegradationAppearanceSuitable Method | | |
|---|---|---|
| Blur | Loss of sharpness | Wiener / inverse filtering |
| Gaussian noise | Random small variations | Gaussian / Wiener filtering |
| Salt-and-pepper | Black and white dots | Median filtering |

### Conclusion

Image enhancement focuses on making an image **look better or reveal useful information**, whereas restoration focuses on **recovering the original image from a modeled degradation**. The filtering technique should be selected according to the type of degradation present.